

TASK 2

CHATBOT FOR FAQs

Complete Project Task / Mini Project Report

Project Objective: Build a chatbot that answers frequently asked questions by finding the most similar FAQ using NLP and similarity matching.

Topic	FAQ Chatbot using NLP
Technologies	Python, NLTK, Scikit-learn
Matching	TF-IDF + Cosine Similarity
Interface	Simple Chat UI (optional)

Complete implementation, explanation, testing and future enhancements.

1. Introduction

A FAQ Chatbot is an application that automatically answers common user questions about a topic or product. Instead of requiring a human to answer every repeated question, the chatbot stores a collection of frequently asked questions and their answers. When a user enters a question, the system processes the text and finds the FAQ that is most similar to the user's question.

This project uses Natural Language Processing (NLP) to clean and process text. TF-IDF is used to represent the FAQ questions numerically, and cosine similarity is used to identify the closest matching FAQ.

2. Objectives

- Collect and store FAQs with their corresponding answers.
- Preprocess questions using NLP techniques.
- Convert text into numerical features using TF-IDF.
- Compare the user's question with stored FAQ questions.
- Select the most similar FAQ using cosine similarity.
- Display the matching answer as a chatbot response.
- Provide a simple chat interface for user interaction.

3. Requirements

- Python 3.x
- NLTK for basic text preprocessing.
- Scikit-learn for TF-IDF and cosine similarity.
- A collection of FAQ questions and answers.
- Optional: HTML/CSS/JavaScript interface or a Python UI.

4. FAQ Dataset

For demonstration, the chatbot uses FAQs about a **College / Student Support System**. The dataset can easily be replaced with FAQs for a product, company, bank, website, or any other topic.

FAQ Question	Answer
What are the college working hours?	The college works from 9:00 AM to 4:00 PM on regular working days.
How can I contact the college?	You can contact the college office through the official phone number or email.
How do I apply for admission?	Complete the admission application form and submit the required documents.
Where can I pay my fees?	Fees can be paid through the college fee portal or at the accounts office.
How can I get my ID card?	Contact the student office after completing the required registration process.
How can I get a scholarship?	Check the scholarship office or official scholarship portal for eligibility and application details.

5. System Workflow

User Question → Text Cleaning → Tokenization → TF-IDF Vectorization → Cosine Similarity → Best FAQ → Chatbot Answer

6. How the System Works

- **Input:** The user enters a question in the chatbot.
- **Preprocessing:** The question is converted to lowercase and unnecessary characters are removed.
- **Tokenization:** Text is divided into smaller words/tokens.
- **Vectorization:** FAQ questions are converted into TF-IDF vectors.
- **Similarity:** Cosine similarity compares the user question with every FAQ.
- **Selection:** The FAQ with the highest similarity score is selected.
- **Response:** The corresponding answer is displayed.

7. Python Implementation

The following program implements the complete FAQ matching logic using NLTK and Scikit-learn.

7.1 Install Required Libraries

```
pip install nltk scikit-learn
```

7.2 Complete Python Program

```
import re
import nltk
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Download tokenizer data if required
nltk.download("punkt", quiet=True)

# FAQ dataset
faqs = [
    {
        "question": "What are the college working hours?",
        "answer": "The college works from 9:00 AM to 4:00 PM on regular working days."
    },
    {
        "question": "How can I contact the college?",
        "answer": "You can contact the college office through the official phone number or email."
    },
    {
        "question": "How do I apply for admission?",
        "answer": "Complete the admission application form and submit the required documents."
    },
    {
        "question": "Where can I pay my fees?",
        "answer": "Fees can be paid through the college fee portal or at the accounts office."
    },
    {
        "question": "How can I get my ID card?",
        "answer": "Contact the student office after completing the required registration process."
    },
    {
        "question": "How can I get a scholarship?",
        "answer": "Check the scholarship office or official scholarship portal for eligibility and application details."
    }
]

def clean_text(text):
    text = text.lower()
    text = re.sub(r"^[a-zA-Z0-9\s]", "", text)
    return text.strip()

questions = [clean_text(item["question"]) for item in faqs]

# Convert FAQ questions into TF-IDF vectors
vectorizer = TfidfVectorizer()
faq_vectors = vectorizer.fit_transform(questions)

def get_answer(user_question):
    cleaned = clean_text(user_question)

    if not cleaned:
        return "Please enter a question."

    user_vector = vectorizer.transform([cleaned])

    # Compare user question with all FAQ questions
    scores = cosine_similarity(user_vector, faq_vectors)[0]
    best_index = scores.argmax()
    best_score = scores[best_index]

    # Minimum confidence threshold
    if best_score < 0.20:
        return "Sorry, I could not find a suitable answer."

    return faqs[best_index]["answer"]
```

```
# Chat loop
print("FAQ Chatbot: Hello! Ask me a question.")
print("Type 'exit' to stop.")

while True:
    user_question = input("You: ")

    if user_question.lower() == "exit":
        print("Bot: Goodbye!")
        break

    print("Bot:", get_answer(user_question))
```

8. Explanation of Important Parts

- **clean_text()**: Converts text to lowercase and removes unnecessary characters.
- **TfidfVectorizer**: Converts FAQ questions into numerical TF-IDF vectors.
- **cosine_similarity()**: Measures how similar the user's question is to each stored FAQ.
- **best_index**: Identifies the FAQ with the highest similarity score.
- **Threshold**: Prevents the chatbot from giving an unrelated answer when similarity is too low.

9. Optional Chat UI

A simple web interface can be added so that users do not have to use the Python terminal. The interface can contain a chat area, text box and Send button.

9.1 HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>FAQ Chatbot</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="chatbox">
    <h2>FAQ Chatbot</h2>
    <div id="messages"></div>

    <div class="input-area">
      <input id="question" placeholder="Ask a question...">
      <button onclick="sendQuestion()">Send</button>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

9.2 CSS

```
body {
  font-family: Arial, sans-serif;
  background: #f4f4f4;
  padding: 40px;
}

.chatbox {
  max-width: 600px;
  margin: auto;
  background: white;
  padding: 20px;
  border-radius: 12px;
}

#messages {
  min-height: 300px;
  border: 1px solid #ddd;
  padding: 15px;
  margin-bottom: 10px;
}

.input-area {
  display: flex;
  gap: 10px;
}

input {
  flex: 1;
  padding: 10px;
}

button {
  padding: 10px 18px;
  cursor: pointer;
}
```

9.3 JavaScript

```
async function sendQuestion() {
  const input = document.getElementById("question");
  const messages = document.getElementById("messages");
  const question = input.value.trim();

  if (!question) return;
```

```

messages.innerHTML += "<p><b>You:</b> " + question + "</p>";

const response = await fetch("/chat", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({question: question})
});

const data = await response.json();

messages.innerHTML += "<p><b>Bot:</b> " + data.answer + "</p>";
input.value = "";
}

```

10. Backend Connection

The optional web UI sends the user's question to a Python backend. The backend runs the FAQ matching function and returns the selected answer as JSON. Flask or FastAPI can be used to create this backend endpoint.

Example API response:

```

{
  "answer": "The college works from 9:00 AM to 4:00 PM on regular working days."
}

```

11. Testing

Test Input	Expected Result
What time does the college open?	Returns the college working-hours answer.
How can I apply for admission?	Returns the admission procedure answer.
Where do I pay college fees?	Returns the fee-payment answer.
Tell me something unrelated to college.	Shows that no suitable FAQ was found when similarity is below the threshold.
Empty input	Displays: Please enter a question.
exit	Ends the chatbot.

12. Advantages

- Automatically answers common questions.
- Reduces repeated manual support work.
- Fast response to users.
- Easy to update by adding new FAQs.
- NLP makes matching more flexible than exact keyword matching.
- Can be integrated into websites or applications.

13. Limitations

- The chatbot can answer only questions related to its FAQ dataset.
- Poorly written or very different questions may produce low similarity.
- The quality of responses depends on the quality and coverage of the FAQ data.
- A larger production system may require stronger intent classification or a more advanced language model.

14. Future Enhancements

- Add a larger FAQ dataset.
- Add automatic language detection and multilingual support.
- Use intent classification for better question understanding.
- Store conversation history.
- Add voice input and text-to-speech.
- Connect the chatbot to a database so FAQs can be updated without changing the code.
- Deploy the chatbot on a website or mobile application.

15. Conclusion

The FAQ Chatbot successfully demonstrates how NLP can be used to automate answers to common questions. The system collects FAQs, cleans and processes text, converts questions into TF-IDF vectors, compares them using cosine similarity, and returns the answer associated with the most similar FAQ. A simple chat interface can be added to make the system easier for users to interact with.

16. Final Project Flow

**Collect FAQs → Clean Text → Tokenize/Preprocess → TF-IDF → Cosine Similarity → Find Best FAQ
→ Display Answer → Continue Chat**